Nova VM

LEARN. PLAY. HAVE FUN!

NOVA VM

Fun & Games

★ ARCADE ACTION!
★ SPACE ADVENTURES!
★ TYPE-IN GAMES!

GAME PROGRAMS™
FOR YOUR NOVA VM

COMPLETE TYPE-IN LISTINGS
FOR HOURS OF RETRO GAMING FUN!
JUST TYPE THEM IN AND PLAY!

Nova Fun & Games

Type-in programs for NovaBASIC and the Nova VM

This book is original Nova VM documentation. It is inspired by the general tradition of early home-computer type-in game books, but it does not reproduce the prose, artwork, screenshots, page designs, or program listings of those books.

The programs in this book are written for NovaBASIC and the Nova VM. The generated disk image for the listings is `fun_n_games.ndi`.

Nova VM and e6502 are open source project names used for this repository.

Typeset with LaTeX and the `lmodern` font family.
Document class: `book`, 11 pt, A4.
Build toolchain: `latexmk` with `pdflatex`.

Dear Programmer

This book is for the kind of person who sees a blinking cursor and wonders what it can do.

Maybe you have written programs before. Maybe this is the first time you have typed a line of BASIC. Either way, you are in the right place. NovaBASIC is a small language with direct access to a lively machine. You can print words, draw pictures, make sounds, move sprites, save files, and build games one line at a time.

The programs in this book are not magic tricks. They are meant to be taken apart. Type them in, run them, change a number, break something, fix it, and then make the game yours.

The Most Important Rule

Do not be afraid of mistakes. A typing error is not a disaster. It is the computer telling you exactly where the next little mystery is.

Nova gives you more screen space, more memory, hardware sprites, colorful graphics, useful sound, and fast helpers for moving data around. The listings start simply because that is how good programs usually start. Once the simple version works, you can make it faster, brighter, stranger, and more your own.

So mount the disk, turn to a listing, and type. The first run might not work. The second or third one probably will. That feeling is worth chasing.

Welcome to Nova Fun & Games.

How To Use This Book

You can use this book in two ways. You can mount `fun_n_games.ndi` and run the programs that are already on the disk, or you can type the listings yourself. Typing is slower, but it teaches you more.

If You Want To Run A Program

Mount `fun_n_games.ndi`, start NovaBASIC, and type:

```
DIR
LOAD "TINY-1"
RUN
```

Use `DIR` when you want to see the programs on the disk. Use `LIST` when you want to see the lines inside a program.

If You Want To Type A Program

Start with `common.bas`. It begins at line 60000 and gives every game a title screen. Type it once, save it, and keep it.

```
NEW
REM TYPE COMMON.BAS HERE
SAVE "COMMON"
```

Then type a game listing. Game listings begin near line 10 and call the common framework with `GOSUB 60000`.

Type-In Note

NovaBASIC only cares about the first two characters of a variable name. This book uses short names on purpose. `SC` is a good score variable. `SCORE` looks nicer, but NovaBASIC treats it like `SC`.

How To Check Your Typing

When a program will not run, do not erase it and start over. Check it in small pieces.

1. Use `LIST` to look at the lines near the error.
2. Check every quote mark in a `PRINT` line.
3. Check every comma, colon, and parenthesis.
4. Make sure `FOR` has a matching `NEXT`.
5. Make sure `GOSUB` has a matching `RETURN`.

Make It Yours

After a program works, change it. Try a new color, a different sound, a faster enemy, or a bigger score. You learn the machine by giving it your own ideas.

For Grownups

This book is written for curious young programmers, but it is also meant to be comfortable for parents, teachers, clubs, and anyone helping from the side of the keyboard.

The programs are intentionally small. A short listing gives a child a real win: they can type the whole thing, run it, and point to the lines that make it work. The point is not to hide the machine. The point is to make the machine friendly enough that a beginner wants to keep going.

A Good Session

A good session is not measured by how many pages were finished. A good session might be:

- typing ten lines without rushing,
- finding one mistake,
- changing a color and seeing the result,
- making a silly sound effect,
- explaining what one variable does.

Helping Without Taking Over

When something breaks, ask small questions first. What line did NovaBASIC mention? What changed since the last run? Does the line have the same number of quote marks on both sides? This keeps the keyboard in the learner's hands.

Why BASIC?

BASIC is immediate. A beginner can type a line, run it, and see a result without learning a build system first. NovaBASIC keeps that direct feeling but adds modern Nova VM hardware for graphics, sprites, sound, and storage.

The style of this book is old-fashioned on purpose: listings, line numbers, experiments, and plain explanations. The goal is not nostalgia by itself. The goal is a path from curiosity to control.

Contents

CHAPTER 1

Before You Type

Nova Fun & Games is a book of small programs for NovaBASIC. It is meant to feel like a program book you can work through at the keyboard: type a little, run a little, change a little, and learn the machine by making it move.

This first volume sets up the shared framework and the first Nova-native listings. Some programs are still short starter listings, but the shape is the same: type a small program, run it, and then change it until it feels like yours.

1.1 The Disk

The listings live on `fun_n_games.ndi`. The disk has no autoboot program. Mount it, load a program, and run it from BASIC:

```
LOAD "ROADHOG"
RUN
```

The image also includes `common.bas`, the shared code used by every program in the book. The generated disk copies the common framework into each game file so a reader can load and run a game directly. The separate `common.bas` file is still present for the type-in workflow.

Type-In Note

If you are typing from the printed book, type `common.bas` once and save it. Then type a game listing below it, starting at line 10. The game will call line 60000 when it wants the common title screen.

1.2 Variable Names

NovaBASIC accepts longer variable names, but only the first two characters are significant. In this book, program variables use one or two characters.

The shared framework owns names beginning with `Z`. Game listings should avoid `Z*` variables unless they are setting the framework controls: `ZT$`, `ZA$`, `ZS$`, and `ZP`.

1.3 Line Numbers

The book uses a simple line-number map:

`10-9999`	Game code typed for the individual program.
`50000-59999`	Reserved for later helper routines local to a large game.
`60000-60280`	Shared Nova Fun & Games common framework.

This keeps the program easy to list on screen and easy to repair after a typing mistake.

CHAPTER 2

The Common Framework

Every program in this book begins by setting a few Z variables and calling line 60000:

```
ZT$="ROADHOG":ZA$="NOVA PORT STUB":ZS$="CROOKED ROAD DRIVING":ZP=1
GOSUB 60000
```

The common framework draws the opening screen, plays a short sound, waits for the player, clears the machine state, and returns to the game.

2.1 Framework Controls

ZT$	Title printed in the center of the splash screen.
ZA$	Author or program-series line.
ZS$	Short subtitle explaining the program.
ZP	Prompt mode. Use 0 for Enter or Space, 1 for Space start.

2.2 What It Does

The framework deliberately keeps to visible, beginner-friendly work:

- It clears Copper state and hides all hardware sprites.
- It enters graphics mode and draws a high-color title frame.
- It centers colorful bitmap titles with GTEXT, LEN, and INT.
- It plays a short sound cue.
- It waits for Enter or Space depending on ZP.
- It returns to text mode and hands control back to the game.

Nova Note

The framework is ordinary NovaBASIC. It is not a hidden ROM routine. Readers can list it, change it, or replace it with their own house style.

2.3 Why This Belongs in the Book

A shared framework gives every program a little polish without making every listing repeat the same title-screen work. It also lets the book teach a real pattern: put common code in one place, keep its Z variables for itself, and call it from the individual programs.

Type-In Note

Keep program-specific code out of `common.bas`. Sprites, maps, levels, and special routines belong in the program that uses them, so each listing can be studied and changed on its own.

CHAPTER 3

Tiny Programs

The first five listings are small demonstrations. Their job is not to be complete games. Their job is to teach the rhythm of NovaBASIC and give readers something they can finish quickly.

3.1 Demo Plan

File	Topic	Nova version
tiny-1.bas	Sprites	Bounce one hardware sprite with gravity, frame timing, and sound.
tiny-2.bas	Sound	Play simple effects and a short musical idea with Nova's sound commands.
tiny-3.bas	Line Draw	Draw points, lines, and patterns on the 320 by 200 graphics display.
tiny-4.bas	Vsprites	Move a larger software sprite so readers see how Nova goes beyond 16 by 16 hardware shapes.
tiny-5.bas	Copper	Change display colors while the screen is being drawn.

3.2 Teaching Rule

Each tiny program should be short enough to type in one sitting. If a feature needs a long setup, it belongs in a later game.

Type-In Note

For printed listings, keep the first demo programs deliberately plain. A reader should be able to type one, run it, and understand why it works without needing the full hardware reference guide.

3.3 Tiny-1: Gravity Ball

What You'll See

TINY-1 puts a shiny ball on the screen, lets gravity pull it down, and bounces it back up from the border. The program is short, but it already feels alive because Nova's sprite hardware, frame timing, and sound commands are all working together.

What It Teaches

- A hardware sprite can move without redrawing the whole screen.
- VSYNC keeps the motion tied to the video frame.
- DY=DY+1 is enough to make simple gravity.
- SOUND adds feedback without any POKEs.

The Listing

```
50 MODE 3:COLOR 1,0,14:GCLS:GOSUB 1000
60 XL=0:XR=304:YT=0:YB=184:BV=14
65 X=40:Y=20:DX=1:DY=0
70 SPRITE 0,ON
80 SPRITE 0,X,Y:VSYNC:VSYNC
90 GET K:IF K=81 OR K=113 THEN 190
100 DY=DY+1:NX=X+DX:NY=Y+DY
120 IF NX<XL THEN NX=XL:DX=-DX:SOUND 50,2
130 IF NX>XR THEN NX=XR:DX=-DX:SOUND 50,2
140 IF NY<YT THEN NY=YT:DY=0:SOUND 60,2
150 IF NY>YB THEN NY=YB:DY=-BV:SOUND 72,3
160 X=NX:Y=NY
170 GOTO 80
190 SPRITE 0,OFF:MODE 0:COLOR 1,0,0:CLS:END
1000 REM LOAD SHINY BALL SPRITE
1010 FOR R=0 TO 15:READ A,B,C,D,E,F,H,J
1020 SPRITEDATA 0,R,A,B,C,D,E,F,H,J:NEXT R
1030 SPRITESHAPE 0,0:SPRITESET 0,6,2:SPRITESET 0,7,0:RETURN
1040 DATA $00,$00,$0F,$FF,$FF,$00,$00,$00
1050 DATA $00,$00,$FF,$FF,$FF,$FF,$00,$00
1060 DATA $00,$0F,$FF,$FF,$FF,$FF,$F0,$00
1070 DATA $00,$FF,$FF,$FF,$EE,$EE,$FF,$00
1080 DATA $0F,$FF,$FF,$EE,$EE,$EE,$EE,$F0
1090 DATA $FF,$FF,$EE,$EE,$EE,$EE,$EE,$FF
1100 DATA $FF,$FE,$EE,$EE,$EE,$EE,$EE,$6F
1110 DATA $FF,$EE,$EE,$EE,$EE,$EE,$66,$6F
1120 DATA $FE,$EE,$EE,$EE,$EE,$66,$66,$6F
1130 DATA $FE,$EE,$EE,$EE,$66,$66,$66,$BF
1140 DATA $FE,$EE,$EE,$66,$66,$66,$BB,$BF
1150 DATA $FF,$EE,$E6,$66,$66,$BB,$BB,$FF
1160 DATA $0F,$EE,$E6,$66,$BB,$BB,$BB,$F0
1170 DATA $00,$FE,$E6,$6B,$BB,$BB,$BF,$00
```

```
1180 DATA $00,$0F,$66,$BB,$BB,$BB,$F0,$00
1190 DATA $00,$0B,$BB,$BB,$BB,$BB,$B0,$00
```

How It Works

Line 50 switches to graphics mode, paints the border, clears the screen, and loads the ball sprite. Line 60 names the four walls. The screen is 320 pixels wide and 200 pixels tall. Because the ball is 16 by 16 pixels, the right wall is `304` and the bottom wall is `184`.

Line 100 is the whole gravity trick. Add 1 to `DY`, and the ball falls faster. Then add `DX` to `X`, add `DY` to `Y`, and the ball moves.

Lines 120 through 150 keep the ball inside the screen. If the next step would cross a wall, the program puts the ball exactly on that wall. At the floor it sets `DY=-BV`, which sends the ball upward again with the same bounce every time. The bottom bounce uses a higher `SOUND` note than the side bounce, so you can hear the collision without adding any POKEs.

Try This

Change `BV` on line 60. A larger number makes the ball bounce higher. A smaller number makes it bounce lower. Then change `DX` on line 65 to make the ball drift sideways faster or slower.

Nova Note

`SPRITE 0,X,Y` only moves the sprite registers. The picture of the ball is already stored in sprite shape memory, so Nova does not have to redraw the ball every frame. That is why a small BASIC loop can still look smooth.

The ball itself is also a Nova lesson. Lines 1000 through 1190 load one 16 by 16 sprite shape. The `DATA` values are the pixels. They use white, light grey, light blue, blue, and dark grey to fake a shiny round surface.

CHAPTER 4

Game Catalog

This catalog records the Nova-native direction for each program on the disk. The goal is not to clone another machine's listing. The goal is to use the same small-program spirit while leaning into Nova hardware.

File	Kind	Nova focus	Direction
roadhog.bas	Driving	Copper road, sprites	Curving road, smooth traffic, better pacing.
piano.bas	Music	SID voices	Keyboard piano with simple recording.
bets.bas	Odds	Text UI	Fast betting table with clear money display.
safe.bas	Puzzle	Randomness	Combination puzzle with sound feedback.
boswain.bas	Action	Sprites	Deck-side movement and hazards.
hanoi.bas	Puzzle	Graphics	Animated disk moves using rectangles.
miser.bas	Strategy	Text	Small resource game with readable turns.
mad.bas	Word play	Strings	Fill-in story with cleaner prompts.
godzilla.bas	Arcade	Sprites, sound	City stomper with destructible blocks.
ratrun.bas	Maze	Tile graphics	Fast maze chase on the larger Nova screen.
yahtzee.bas	Dice	Text UI	Dice boxes, scoring grid, optional sound.
leap.bas	Timing	Keyboard	One-button timing challenge.
box.bas	Puzzle	Graphics	Sliding or packing puzzle with visible board.
lawn.bas	Action	Sprites	Mow a field while avoiding moving hazards.
kalah.bas	Board	Text/graphics	Board layout with counters and hints.
bonzo.bas	Arcade	Sprites	Character movement and simple animation.
bjack.bas	Cards	Text UI	Clean blackjack table and betting.
fire.bas	Action	Sprites	Fire control with expanding danger.
zap.bas	Shooter	Sprites, sound	Quick target game with hardware sprites.
everest.bas	Climb	Graphics	Better mountain, weather, and route logic.

File	Kind	Nova focus	Direction
reversi.bas	Board	Graphics	8 by 8 board with legal-move highlighting.
bop.bas	Reflex	Sound	Sequence or timing game with tones.
spot.bas	Memory	Graphics	Pattern matching on colored cells.
dots.bas	Board	Lines	Dots-and-boxes with a clear grid.
capture.bas	Strategy	Graphics	Territory game with fast redraws.
dive.bas	Arcade	Sprites	Diving motion with water and scoring.
stop.bas	Reflex	Timing	Stop-the-marker with frame-based timing.
piegram.bas	Graphics	Shapes	Chart drawing and angle demos.
bat.bas	Arcade	Sprites	Bat and ball motion using collision checks.
rescue.bas	Action	Sprites	Rescue mission with scrolling pressure.

4.1 Common Shape

Every game chapter should follow the same shape:

1. What the player sees.
2. What Nova hardware the program uses.
3. The listing.
4. How the listing works.
5. Experiments to try.

That makes the book friendly for readers and maintainable for us.

CHAPTER 5

Making It Nova

NovaBASIC can act like a simple home-computer BASIC, but it is sitting on a machine with useful hardware. The programs in this book should use that hardware when it makes the game clearer or more exciting.

5.1 Graphics

Use the graphics screen for boards, maps, roads, mountains, and charts. A larger screen means old cramped layouts can breathe. Prefer simple readable geometry over decorative clutter.

5.2 Sprites

Use sprites for objects that move every frame: cars, players, enemies, balls, and cursors. If an object is larger than 16 by 16, use a metasprite helper once that library is stable and available from BASIC.

5.3 Sound

Small sound cues matter. A short note for a correct move, a lower note for a mistake, and a short tune on the title screen can make a type-in game feel finished without making the listing hard to understand.

5.4 Expanded Memory

Use XRAM for tables, levels, and generated data when normal BASIC memory would make the program awkward. Keep the first versions simple, then move bulky data out only when the game needs it.

5.5 The Rule

If a Nova feature makes a game clearer, use it. If it only makes the listing harder to type, save it for an advanced version.

CHAPTER A

Common Listing

This is the shared framework used by every program in the book.

```
60000 REM NOVA FUN & GAMES COMMON FRAMEWORK
60010 IF ZT$="" THEN ZT$="NOVA FUN & GAMES"
60020 IF ZA$="" THEN ZA$="NOVA BASIC"
60030 IF ZS$="" THEN ZS$="TYPE-IN GAME FRAMEWORK"
60040 IF ZP<>1 THEN ZP=0
60050 COPPER OFF:COPPER CLEAR
60060 FOR ZI=0 TO 15:SPRITE ZI,OFF:NEXT ZI
60070 MODE 3:COLOR 1,6,0:CLS:GCLS
60080 FOR ZI=0 TO 15
60090 ZC=(ZI AND 7)+8:GCOLOR ZC
60100 LINE 0,ZI*12,319,199-ZI*12
60110 NEXT ZI
60120 GCOLOR 11:FILL 16,30,303,162
60130 GCOLOR 1:RECT 16,30,303,162:GCOLOR 14:RECT 22,36,297,156
60140 GCOLOR 7:LINE 24,44,295,44:GCOLOR 10:LINE 24,146,295,146
60150 ZL=LEN(ZT$):ZM=2:IF ZL>18 THEN ZM=1
60160 ZX=INT((320-ZL*8*ZM)/2):IF ZX<4 THEN ZX=4
60170 GCOLOR 11:GTEXT ZX+2,62,0,ZM,ZT$:GCOLOR 14:GTEXT ZX,60,0,ZM,ZT$
60180 ZL=LEN(ZA$):ZX=INT((320-ZL*8)/2):GCOLOR 1:GTEXT ZX,96,0,1,ZA$
60190 ZL=LEN(ZS$):ZX=INT((320-ZL*8)/2):GCOLOR 13:GTEXT ZX,112,0,1,ZS$
60200 GCOLOR 7:GTEXT 64,136,0,1,"COMMON FRAMEWORK AT 60000"
60210 IF ZP=1 THEN ZW$="PRESS SPACE" ELSE ZW$="PRESS ENTER"
60220 ZL=LEN(ZW$):ZX=INT((320-ZL*8)/2):GCOLOR 15:GTEXT ZX,152,0,1,ZW$
60230 VOLUME 12:SOUND 60,5:SOUND 67,5:SOUND 72,8
60240 GET ZK
60245 IF ZK=0 THEN 60240
60250 IF ZP=1 AND ZK<>32 AND ZK<>13 THEN 60240
60260 IF ZP=0 AND ZK<>13 AND ZK<>32 THEN 60240
60270 MODE 0:GCLS:COLOR 1,0,0:CLS
60280 RETURN
```

Listing A.1: common.bas

CHAPTER B

Starter Listings

These are the starter listings currently stored on `fun_n_games.ndi`. Each one calls the common framework and leaves a clear place for the full game code. The short introductions are here to answer the most important question before anyone starts typing: why is this program worth bringing to life?

B.1 Tiny 1

Why Type This In?

This is the quickest way to see NovaBASIC make something feel alive. A shiny hardware sprite falls, speeds up under gravity, rebounds inside the visible screen border, and waits for the video frame so the motion feels smooth instead of jumpy. In just a few lines, you get physics, sprites, sound, and timing working together.

```
10 REM TINY-1 - GRAVITY BALL
20 REM HARDWARE SPRITE WITH GRAVITY
30 ZT$="GRAVITY BALL":ZA$="NOVA FUN & GAMES":ZS$="A TINY PHYSICS DEMO":ZP=0
40 GOSUB 60000
50 MODE 3:COLOR 1,0,14:GCLS:GOSUB 1000
60 XL=0:XR=304:YT=0:YB=184:BV=14
65 X=40:Y=20:DX=1:DY=0
70 SPRITE 0,ON
80 SPRITE 0,X,Y:VSYNC:VSYNC
90 GET K:IF K=81 OR K=113 THEN 190
100 DY=DY+1:NX=X+DX:NY=Y+DY
120 IF NX<XL THEN NX=XL:DX=-DX:SOUND 50,2
130 IF NX>XR THEN NX=XR:DX=-DX:SOUND 50,2
140 IF NY<YT THEN NY=YT:DY=0:SOUND 60,2
150 IF NY>YB THEN NY=YB:DY=-BV:SOUND 72,3
160 X=NX:Y=NY
170 GOTO 80
190 SPRITE 0,OFF:MODE 0:COLOR 1,0,0:CLS:END
1000 REM LOAD SHINY BALL SPRITE
1010 FOR R=0 TO 15:READ A,B,C,D,E,F,H,J
1020 SPRITEDATA 0,R,A,B,C,D,E,F,H,J:NEXT R
1030 SPRITESHAPE 0,0:SPRITESET 0,6,2:SPRITESET 0,7,0:RETURN
1040 DATA $00,$00,$0F,$FF,$FF,$00,$00,$00
1050 DATA $00,$00,$FF,$FF,$FF,$FF,$00,$00
1060 DATA $00,$0F,$FF,$FF,$FF,$FF,$F0,$00
1070 DATA $00,$FF,$FF,$FF,$EE,$EE,$FF,$00
1080 DATA $0F,$FF,$FF,$EE,$EE,$EE,$EE,$F0
1090 DATA $FF,$FF,$EE,$EE,$EE,$EE,$EE,$FF
1100 DATA $FF,$FE,$EE,$EE,$EE,$EE,$EE,$6F
1110 DATA $FF,$EE,$EE,$EE,$EE,$EE,$66,$6F
1120 DATA $FE,$EE,$EE,$EE,$EE,$66,$66,$6F
1130 DATA $FE,$EE,$EE,$EE,$66,$66,$66,$BF
1140 DATA $FE,$EE,$EE,$66,$66,$66,$BB,$BF
1150 DATA $FF,$EE,$E6,$66,$66,$BB,$BB,$FF
1160 DATA $0F,$EE,$E6,$66,$BB,$BB,$BB,$F0
1170 DATA $00,$FE,$E6,$6B,$BB,$BB,$BF,$00
1180 DATA $00,$0F,$66,$BB,$BB,$BB,$F0,$00
1190 DATA $00,$0B,$BB,$BB,$BB,$BB,$B0,$00
```

Listing B.1: Tiny 1 starter listing

B.2 Tiny 2

Why Type This In?

Color is one of the easiest ways to make a small program feel bigger than it is. This tiny demo is about playing with Nova's bright display modes and learning how a few color choices can turn plain text or simple shapes into something that feels like a real screen from a game.

```
10 REM TINY-2 - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="TINY-2":ZA$="NOVA PORT STUB":ZS$="PATTERN AND COLOR DEMO":ZP=0
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "TINY-2 PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.2: Tiny 2 starter listing

B.3 Tiny 3

Why Type This In?

Before a game has sprites, it needs marks on the screen. This listing invites you to draw lines, dots, boxes, and patterns on Nova's larger graphics display, then start asking the fun questions: what happens if the computer draws faster, brighter, or in a different direction?

```
10 REM TINY-3 - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="TINY-3":ZA$="NOVA PORT STUB":ZS$="PATTERN AND COLOR DEMO":ZP=0
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "TINY-3 PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.3: Tiny 3 starter listing

B.4 Tiny 4

Tiny 4

TYPE-IN LISTING

Why Type This In?

A silent computer feels asleep. This program wakes Nova up with tones, pauses, and little musical ideas, showing how sound can make even a tiny listing feel playful. Type it in, change a number, and you can hear the machine answer back.

```
10 REM TINY-4 - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="TINY-4":ZA$="NOVA PORT STUB":ZS$="PATTERN AND COLOR DEMO":ZP=0
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "TINY-4 PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.4: Tiny 4 starter listing

B.5 Tiny 5

Why Type This In?

This is the first step toward arcade motion. A hardware sprite can move without redrawing the whole screen, which means NovaBASIC can keep the program simple while the video hardware does the busy work. Once one sprite moves, ships, balls, bats, monsters, and players are all within reach.

```
10 REM TINY-5 - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="TINY-5":ZA$="NOVA PORT STUB":ZS$="PATTERN AND COLOR DEMO":ZP=0
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "TINY-5 PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.5: Tiny 5 starter listing

B.6 Roadhog

Roadhog

TYPE-IN LISTING

Why Type This In?

Roadhog is for anyone who wants the screen to feel like it is moving under the player. Nova's display hardware can handle colorful roads, smooth sprite traffic, and timed updates, so this starter points toward a driving game that feels faster and cleaner than old type-in road games ever could.

```
10 REM ROADHOG - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="ROADHOG":ZA$="NOVA PORT STUB":ZS$="CROOKED ROAD DRIVING":ZP=1
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "ROADHOG PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.6: Roadhog starter listing

B.7 Piano

Why Type This In?

This is a program you can play even before it becomes a game. The idea is simple: turn the keyboard into an instrument, let Nova's sound hardware carry the tune, and then experiment with rhythm, pitch, and recording. It is a friendly doorway into making the machine sing.

```
10 REM PIANO - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="PIANO":ZA$="NOVA PORT STUB":ZS$="SCREEN KEYBOARD MUSIC":ZP=0
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "PIANO PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.7: Piano starter listing

B.8 Bets

Why Type This In?

Bets turns numbers into suspense. It is a good place to learn how a game can feel exciting with text, money, odds, and quick feedback instead of lots of graphics. Nova makes the table clean and fast, so the player can focus on the next risky choice.

```
10 REM BETS - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="BETS":ZA$="NOVA PORT STUB":ZS$="UNUSUAL CARD GAME":ZP=0
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "BETS PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.8: Bets starter listing

B.9 Safe

Why Type This In?

A safe-cracking puzzle is small, but it has everything a good program needs: mystery, memory, feedback, and a reason to try again. Nova can add color and sound to every guess, turning a few numbers into a tense little machine on the screen.

```
10 REM SAFE - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="SAFE":ZA$="NOVA PORT STUB":ZS$="BURGLARIZE THE SAFE":ZP=1
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "SAFE PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.9: Safe starter listing

B.10 Boswain

Boswain

TYPE-IN LISTING

Why Type This In?

Boswain aims for action on a busy deck. The fun is in steering through trouble while the screen shows clear movement and the sound gives each mistake some bite. Nova's sprites make it a natural fit for a fast nautical challenge instead of a slow text-only exercise.

```
10 REM BOSWAIN - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="BOSWAIN":ZA$="NOVA PORT STUB":ZS$="WATCH HIS HANDS":ZP=1
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "BOSWAIN PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.10: Boswain starter listing

B.11 Hanoi

Why Type This In?

Tower of Hanoi is a classic because the rules are tiny and the puzzle grows in your head. Nova lets the program draw the disks, move them, and make each step visible, so a brain teaser becomes something you can watch, solve, and improve.

```
10 REM HANOI - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="HANOI":ZA$="NOVA PORT STUB":ZS$="MOVE THE GOLDEN DISKS":ZP=0
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "HANOI PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.11: Hanoi starter listing

B.12 Miser

Miser

TYPE-IN LISTING

Why Type This In?

Miser is about squeezing choices out of limited resources. It is the kind of game where a plain number can start to feel precious, and NovaBASIC is perfect for making those turns readable, fast, and easy to change into your own strategy game.

```
10 REM MISER - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="MISER":ZA$="NOVA PORT STUB":ZS$="HAUNTED TREASURE HUNT":ZP=0
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "MISER PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.12: Miser starter listing

B.13 Mad

Why Type This In?

Mad is proof that games do not need explosions to be fun. With strings, prompts, and a little surprise, Nova can become a story machine that makes people laugh at what they typed a minute earlier. It is also a gentle way to learn how programs handle words.

```
10 REM MAD - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="MAD":ZA$="NOVA PORT STUB":ZS$="STORY WORD PLAY":ZP=0
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "MAD PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.13: Mad starter listing

B.14 Godzilla

Godzilla

TYPE-IN LISTING

Why Type This In?

This one should feel big, loud, and a little ridiculous. Nova's sprites and graphics can turn a simple city grid into something stompable, with buildings to smash and sound effects to make the damage feel satisfying. It is a great excuse to make the computer misbehave on purpose.

```
10 REM GODZILLA - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="GODZILLA":ZA$="NOVA PORT STUB":ZS$="MONSTER RAMPAGE":ZP=1
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "GODZILLA PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.14: Godzilla starter listing

B.15 Ratrun

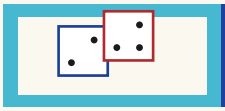
Why Type This In?

Maze games are where speed and clarity matter. Ratrun points toward quick tile drawing, crisp movement, and a chase that uses Nova's larger screen to show more of the maze at once. Type it in and you are halfway to building your own arcade labyrinth.

```
10 REM RATRUN - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="RATRUN":ZA$="NOVA PORT STUB":ZS$="RAT'S-EYE MAZE":ZP=1
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "RATRUN PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.15: Ratrun starter listing

B.16 Yahtzee



Yahtzee

TYPE-IN LISTING

Why Type This In?

Dice games are perfect for learning how a program manages turns, choices, and score sheets. Nova can make the dice and scoring grid easy to read, while BASIC keeps the rules close enough that you can tinker with them without getting lost.

```
10 REM YAHTZEE - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="YAHTZEE":ZA$="NOVA PORT STUB":ZS$="FIVE-DICE SCORE GAME":ZP=0
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "YAHTZEE PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.16: Yahtzee starter listing

B.17 Leap

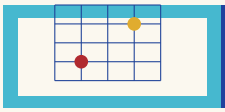
Why Type This In?

Leap is a one-button kind of idea, which makes it dangerous in the best way. The player waits, jumps, misses, tries again, and soon the timing gets personal. Nova's frame timing and sprite motion can make that simple loop feel sharp and fair.

```
10 REM LEAP - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="LEAP":ZA$="NOVA PORT STUB":ZS$="PEG-JUMP PUZZLE":ZP=0
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "LEAP PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.17: Leap starter listing

B.18 Box



Box
TYPE-IN LISTING

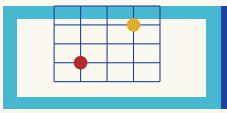
Why Type This In?

Box is for puzzle builders. A board, a few pieces, and one clever rule can keep a player busy for ages. Nova's graphics make the board visible and friendly, while the listing stays small enough that you can redesign the puzzle after you understand it.

```
10 REM BOX - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="BOX":ZA$="NOVA PORT STUB":ZS$="RAY-TRACE HIDDEN ATOMS":ZP=0
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "BOX PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.18: Box starter listing

B.19 Lawn



Lawn

TYPE-IN LISTING

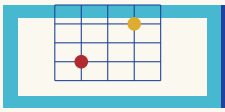
Why Type This In?

Lawn starts with a silly promise: mow the field, do not get clobbered. That gives you a screen that changes as you play, moving hazards, and a clear goal. Nova's sprites and fast redraws can make the grass, mower, and trouble easy to follow.

```
10 REM LAWN - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="LAWN":ZA$="NOVA PORT STUB":ZS$="MOW BEFORE GAS RUNS OUT":ZP=1
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "LAWN PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.19: Lawn starter listing

B.20 Kalah



Kalah

TYPE-IN LISTING

Why Type This In?

Kalah is an ancient-feeling board game with modern programming lessons hiding inside it. You get pits, stones, turns, captures, and simple strategy. Nova can draw the board and counters clearly, making it easier to understand both the game and the code.

```
10 REM KALAH - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="KALAH":ZA$="NOVA PORT STUB":ZS$="SOW PEBBLES TO WIN":ZP=0
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "KALAH PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.20: Kalah starter listing

B.21 Bonzo

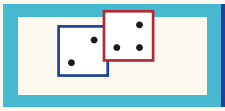
Why Type This In?

Bonzo is a place for character. Once a little figure moves on screen, every added frame, sound, and obstacle makes it feel more like a real arcade game. Nova's sprite hardware lets BASIC spend less time drawing and more time deciding what Bonzo should do next.

```
10 REM BONZO - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="BONZO":ZA$="NOVA PORT STUB":ZS$="RACE TO THE TOP":ZP=1
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "BONZO PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.21: Bonzo starter listing

B.22 Bjack



Bjack

TYPE-IN LISTING

Why Type This In?

Blackjack makes arithmetic feel like risk. The program deals, the player decides, and every card changes the mood of the hand. Nova can keep the table readable and lively, with enough polish that a text-and-card game still feels like a night at the arcade counter.

```
10 REM BJACK - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="BJACK":ZA$="NOVA PORT STUB":ZS$="DEAL ANOTHER HAND":ZP=0
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "BJACK PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.22: Bjack starter listing

B.23 Fire

Why Type This In?

Fire is about pressure. Something is spreading, time is short, and the player has to act before the screen gets out of control. Nova's color, sprites, and sound can make danger visible right away, which is exactly what an action listing needs.

```
10 REM FIRE - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="FIRE":ZA$="NOVA PORT STUB":ZS$="STOP THE INFERNO":ZP=1
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "FIRE PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.23: Fire starter listing

B.24 Zap

Zap

TYPE-IN LISTING

Why Type This In?

Zap is pure reflex fun: see the target, move fast, fire, and hear the result. Hardware sprites are made for this kind of quick action, and Nova lets BASIC build a target game that feels snappy without burying the reader under complicated drawing code.

```
10 REM ZAP - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="ZAP":ZA$="NOVA PORT STUB":ZS$="ARENA ZAPPER ACTION":ZP=1
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "ZAP PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.24: Zap starter listing

B.25 Everest

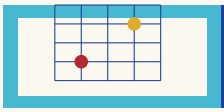
Why Type This In?

Everest should feel like a climb, not just a row of numbers. Nova's graphics can show the mountain, the route, the weather, and the danger, while the program turns each decision into another step upward. It is a survival story you can type.

```
10 REM EVEREST - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="EVEREST":ZA$="NOVA PORT STUB":ZS$="CLIMB THE HIGHEST PEAK":ZP=1
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "EVEREST PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.25: Everest starter listing

B.26 Reversi



Reversi

TYPE-IN LISTING

Why Type This In?

Reversi is a quiet game that becomes fierce once the board starts flipping. This listing direction shows how Nova can make a clean 8 by 8 board, highlight moves, and redraw pieces fast enough that strategy stays in the foreground.

```
10 REM REVERSI - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="REVERSI":ZA$="NOVA PORT STUB":ZS$="STRATEGY AND GUILE":ZP=0
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "REVERSI PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.26: Reversi starter listing

B.27 Bop

Bop

TYPE-IN LISTING

Why Type This In?

Bop is all about rhythm and reaction. A beep, a flash, a pause, and suddenly the player is leaning toward the keyboard waiting for the next cue. Nova's sound and timing make it a perfect little laboratory for building games that feel musical.

```
10 REM BOP - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="BOP":ZA$="NOVA PORT STUB":ZS$="COUNTING WITH STYLE":ZP=0
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "BOP PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.27: Bop starter listing

B.28 Spot

Spot

TYPE-IN LISTING

Why Type This In?

Spot turns the screen into a memory and observation test. Colored cells, quick changes, and simple scoring can create a surprising amount of tension. Nova's bright display makes patterns easy to see, which means the challenge can come from the game instead of the screen being muddy.

```
10 REM SPOT - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="SPOT":ZA$="NOVA PORT STUB":ZS$="FOUR SQUARES IN A ROW":ZP=0
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "SPOT PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.28: Spot starter listing

B.29 Dots

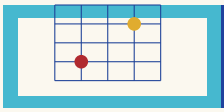
Why Type This In?

Dots starts as a school-paper game and becomes a programming lesson in lines, boxes, and turns. Nova can draw the grid cleanly, update only what changed, and make each captured square feel earned. It is a small game with a very visible payoff.

```
10 REM DOTS - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="DOTS":ZA$="NOVA PORT STUB":ZS$="BOXES AGAINST NOVA":ZP=0
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "DOTS PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.29: Dots starter listing

B.30 Capture



Capture

TYPE-IN LISTING

Why Type This In?

Capture is for players who like taking over space. A board game about territory gives NovaBASIC a chance to show fast redraws, clear ownership, and computer-managed rules. Once the board is on screen, it begs for new strategies and house rules.

```
10 REM CAPTURE - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="CAPTURE":ZA$="NOVA PORT STUB":ZS$="CHASED BY TWO BEASTS":ZP=1
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "CAPTURE PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.30: Capture starter listing

B.31 Dive

Why Type This In?

Dive should feel like motion through water: timing the drop, judging the path, and chasing a better score. Nova's sprites can handle the diver while graphics make the pool or sea feel alive, giving a small arcade idea room to splash around.

```
10 REM DIVE - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="DIVE":ZA$="NOVA PORT STUB":ZS$="RETRIEVE SUNKEN TREASURE":ZP=1
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "DIVE PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.31: Dive starter listing

B.32 Stop

Stop

TYPE-IN LISTING

Why Type This In?

Stop is the kind of program that looks easy until you try to beat it. A marker moves, your timing matters, and the machine judges you instantly. Nova's frame-based timing keeps the challenge honest, so improving feels like skill rather than luck.

```
10 REM STOP - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="STOP":ZA$="NOVA PORT STUB":ZS$="CAPTURE THREE COLUMNS":ZP=0
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "STOP PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.32: Stop starter listing

B.33 Piegram

Why Type This In?

Piegram turns math into a picture. It is not just a chart program; it is a chance to see angles, slices, colors, and labels become something visual. Nova's graphics make it a good first step toward dashboards, gauges, and game status screens.

```
10 REM PIEGRAM - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="PIEGRAM":ZA$="NOVA PORT STUB":ZS$="DODGE INVISIBLE PIES":ZP=1
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "PIEGRAM PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.33: Piegram starter listing

B.34 Bat

Why Type This In?

Bat is the doorway to paddle games. A moving ball, a player-controlled bat, and simple collision checks can become tennis, breakout, or a dozen other arcade ideas. Nova's sprites and timing do the heavy lifting so the listing can stay readable.

```
10 REM BAT - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="BAT":ZA$="NOVA PORT STUB":ZS$="HUNGRY NIGHT FLIGHT":ZP=1
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "BAT PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.34: Bat starter listing

B.35 Rescue

Why Type This In?

Rescue gives the player a job that matters: get in, save someone, and get out before things go wrong. Nova's sprites, scrolling pressure, and sound can make a small mission feel urgent, which is exactly the kind of magic a type-in game should deliver.

```
10 REM RESCUE - FUN & GAMES STUB
20 REM LOAD COMMON FIRST OR USE THE NDI COMBINED COPY
30 ZT$="RESCUE":ZA$="NOVA PORT STUB":ZS$="ALIEN PLANET EXTRACTION":ZP=1
40 GOSUB 60000
50 REM GAME CODE STARTS HERE
60 PRINT "RESCUE PORT STUB"
70 PRINT "REPLACE THIS STUB WITH GAME CODE."
80 PRINT "PRESS Q TO END."
90 GET K:IF K=0 THEN 90
100 IF K<>81 AND K<>113 THEN 90
110 END
```

Listing B.35: Rescue starter listing

CHAPTER C

Reference Card

C.1 Loading the Disk

```
DIR
LOAD "COMMON"
LIST 60000-60280
LOAD "TINY-1"
RUN
```

C.2 Framework Variables

ZT$	Game title.
ZA$	Author or series line.
ZS$	Subtitle.
ZP	Prompt mode: 0 for Enter or Space, 1 for Space.

C.3 Variable Rule

Only the first two characters of a NovaBASIC variable name are significant. Use one- or two-character names in printed listings. Avoid Z* in game code because the common framework owns that prefix.

C.4 Line Map

10	Program setup.
20-9999	Game logic.
60000	Common title framework begins.
60280	Common title framework returns.

When Things Go Wrong

Every programmer spends time fixing programs. That is not the bad part of programming. That is programming.

The Program Stops With An Error

Read the line number first. Then list a few lines around it:

```
LIST 80-120
```

Most beginner bugs are small: a missing quote, a wrong variable, a line number that points to the wrong place, or a `NEXT` without the right `FOR`.

The Program Runs But Looks Wrong

If the program runs but the screen looks strange, check the setup lines. Look for `MODE`, `COLOR`, `GCOLOR`, `GCLS`, and `CLS`. A game that draws on the graphics screen needs the right mode and the right colors before it starts drawing.

The Controls Do Nothing

Look for the `GET` line. A typical key loop looks like this:

```
90 GET K:IF K=0 THEN 90
```

That means “keep waiting until a key appears.” If the program waits forever, check which key codes it accepts after that line.

The Best Debugging Tool

Use `PRINT`. If you do not know what a variable contains, print it. If you do not know whether a line is being reached, print a short message there.

```
210 PRINT "X=";X;" Y=";Y
```

You can remove the extra `PRINT` lines after the program works.

Words To Know

Animation A trick that shows different pictures quickly so something appears to move.

BASIC The language used in this book. NovaBASIC uses numbered lines and commands such as `PRINT`, `GOTO`, and `FOR`.

Bit A tiny piece of information that is either on or off.

Byte Eight bits. Many Nova hardware values fit in one byte.

Command A word that tells BASIC to do something, such as `PRINT` or `LINE`.

Framework Shared program code used by more than one game. In this book, the common framework starts at line 60000.

Graphics screen The part of the display used for drawing pixels, lines, rectangles, circles, sprites, roads, maps, and game boards.

Line number The number at the start of a BASIC line. NovaBASIC uses line numbers to keep the program in order and to jump with `GOTO` or `GOSUB`.

Loop A part of a program that repeats. Games use loops to keep reading keys, moving objects, and redrawing the screen.

Sprite A small hardware-controlled picture that can move around the screen without redrawing the whole background.

String Text stored in a variable. String variable names end with `$`, such as `NM$`.

Variable A named place to store a value. In this book, variable names are usually one or two characters long.

XRAM Nova expansion memory. Bigger programs can use it to store tables, levels, pictures, and other data outside normal BASIC memory.

Where To Go Next

When you finish a program, you are not really finished. A working program is a place to start experimenting.

Five Good Changes

1. Change the colors.
2. Add a sound when something good happens.
3. Add a sound when something bad happens.
4. Make the game a little faster after each point.
5. Add a title screen line with your name on it.

Bigger Challenges

After the small changes, try one bigger change:

- Add a second player.
- Add a high score.
- Add a new enemy pattern.
- Draw a better board or background.
- Save a level or score table to disk.

Keep A Programmer's Notebook

Write down the changes you try. Keep the ones that work. Cross out the ones that do not. A notebook turns random experiments into a map of what you have learned.

One Last Secret

Nobody understands a whole program all at once. Good programmers understand one small piece, then another, then another. That is enough.

TYPE IT.
RUN IT.
CHANGE IT.

Nova Fun & Games is a friendly collection of NovaBASIC type-in programs for curious new programmers.

Inside you will find:

* a shared title-screen framework
* starter listings for every game
* NovaBASIC notes and experiments
* debugging help for first-time coders

Mount `fun_n_games.ndi`, load a program, and start typing. The games begin small, because small programs are the easiest ones to understand, fix, and make your own.